PRACTICAS CURSO DE C AVANZADO PARA LINUX

1) ..
2) Para punteros a función:

   Desarrollar una aplicación en C, ejecutada en Ubuntu/Lubuntu, que implemente una **Máquina de Estados Finitos (FSM)** capaz de reaccionar a distintos **eventos** y cambiar de estado según la lógica definida. La FSM debe estar completamente estructurada utilizando:

   - un **enum de estados**,
   - un **enum de eventos**,
   - una **estructura de contexto**,
   - una **tabla de handlers**,
   - una **función por estado**,
   - una **función de despacho de eventos**,
   - y un **bucle principal** que procese eventos simulados.

   Definir un conjunto de estados que representen el ciclo de vida de un sistema simple:

   - IDLE, RUNNING, ERROR

   Definir un conjunto de eventos que afecten a la FSM:

   - START, STOP, FAIL

   Mantener un contexto, que incluya al menos:

   - El estado actual,
   - un contador o dato auxiliar (opcional).

   Implementar una función manejadora por cada estado, que reciba:

   - el contexto (fsm_t *),
   - el evento (event_t), y devuelva el siguiente estado.

   Crear una tabla de handlers, indexada por estado, que permita saltar directamente a la función correspondiente sin usar switch.

   Implementar una función de despacho de eventos, que:

   - lea el estado actual del contexto,
   - invoque el handler adecuado,

- actualice el estado del contexto.

Simular una secuencia de eventos, por ejemplo:

START → FAIL → START → STOP → START → STOP

Ejecutar la FSM en un bucle, procesando cada evento y mostrando:

- el estado actual,
- el evento recibido,
- el estado resultante.

Salida del programa:

[Estado actual], evento recibido, y el nuevo estado.

[IDLE] Evento: START → Nuevo estado: RUNNING

[RUNNING] Evento: FAIL → Nuevo estado: ERROR

[ERROR] Evento: START → Nuevo estado: RUNNING

[RUNNING] Evento: STOP → Nuevo estado: IDLE

[IDLE] Evento: START →Nuevo estado: RUNNING

[RUNNING] Evento: STOP → Nuevo estado: IDLE

3) PRÁCTICA: PATRÓN STATE CON CALLBACKS EN C

Desarrollar en C una aplicación que simule un gestor de descargas mediante una máquina de estados y el patrón de diseño State.

La descarga podrá encontrarse en uno de los siguientes **estados**:

- PREPARADA
- DESCARGANDO
- PAUSADA
- FINALIZADA
- ERROR

La aplicación deberá procesar los siguientes **eventos**:

- INICIAR
- AVANZAR
- PAUSAR
- REANUDAR
- FALLAR
- CANCELAR
- REINICIAR

COMPORTAMIENTO

- PREPARADA: permite iniciar o cancelar la descarga.
- DESCARGANDO: permite avanzar, pausar, cancelar o provocar un error.
- PAUSADA: permite reanudar o cancelar. El evento AVANZAR deberá ignorarse.
- FINALIZADA: estado terminal alcanzado cuando el progreso llega al 100 %.
- ERROR: permite reiniciar la descarga desde cero.

Cada evento AVANZAR incrementará el progreso de la descarga en un 25 %.

La solución deberá utilizar:

- Enumeraciones para representar los estados y los eventos.
- Una estructura de contexto con el estado actual, el porcentaje descargado y el código de error.
- Una estructura común para representar el comportamiento de los estados.
- Callbacks on_enter, on_event y on_exit.
- Punteros a función para implementar los callbacks.
- Una función para despachar eventos.
- Una función común para realizar las transiciones entre estados.

SECUENCIA DE PRUEBA

La aplicación deberá procesar la siguiente secuencia:

- INICIAR
- AVANZAR
- AVANZAR
- PAUSAR
- AVANZAR
- REANUDAR
- AVANZAR
- AVANZAR

El evento AVANZAR recibido mientras la descarga está en estado PAUSADA deberá ignorarse.

RESULTADO ESPERADO

La aplicación deberá mostrar:

- El estado actual.
- El evento recibido.
- Las transiciones realizadas.
- La ejecución de los callbacks de entrada y salida.
- El porcentaje descargado.
- Los eventos ignorados o no permitidos.

La ejecución deberá terminar cuando la descarga alcance el estado FINALIZADA.